

第一题

试题名称：第二大整数

时间限制： 1.0s

内存限制： 128.0MB

【问题描述】

编写一个程序，读入一组各不相同的整数（不超过 20 个），当用户输入 0 时，表示输入结束。然后程序将从这组整数中，把第二大的那个整数找出来，并把它打印出来。

说明：（1）0 表示输入结束，它本身并不计入在这组整数中。（2）在这组整数中，既有正数，也可能有负数。（3）这组整数的个数不少于 2 个。

【输入描述】

输入有若干行，每一行是一个整数，最后一行是结束标记 0。

【输出描述】

输出只有一行，即在这组整数中，排名第二大的那个数。

【输入样例】

```
100
-100
200
0
```

【输出样例】

```
100
```

第二题

试题名称：字符重排列

时间限制： 1.0s

内存限制： 128.0MB

【问题描述】

输入一个字符串，该字符串只包含三种类型的字符：小写字母、大写字母和空格。然后将这个字符串中的字符重新排列，生成一个新的字符串，即把所有的小写字母放在最前面，所有的空格放在中间，所有的大写字母放在最后。而且这些小写、大写字母原来的顺序不能乱。最后把这个新的字符串打印出来。

【输入描述】

输入一个字符串（长度不超过 100）

【输出描述】

输出字符重排列以后得到的新字符串。

【输入样例】

```
B a Ab
```

【输出样例】

```
ab BA（注意内部有两个空格）
```

第三题

试题名称：跳绳比赛

时间限制： 1.0s

内存限制： 128.0MB

【问题描述】

诚志小学即将举行跳绳比赛，看谁能在一分钟的时间内跳更多次。班主任老师让每一位同学回家后要练习跳绳技能，练习后自己进行一次模拟比赛。

每天晚上学生要在微信群上报模拟比赛的成绩，格式为（学生姓名，成绩），其中成绩即为在一分钟内跳了多少次。有的学生比较积极，每天都会进行练习并上报成绩，而有的学生则由于各种各样的原因，并不是每天都能完成这项任务，甚至有的学生连一次成绩都没有上报。一周过去了，老师请你帮忙编写一个程序，统计班级跳绳的最高成绩和最低成绩，以及参加了练习的学生人数。所谓参加了练习的学生人数，就是说，如果某一位同学上报过一次或多次成绩，那么都计为一人。

【输入描述】

先输入一个正整数 N（其值不超过 100），接下来再输入 N 行，每一行是一个学生的一次成绩，包括姓名和成绩。

【输出描述】

输出 3 个正整数，表示最高成绩、最低成绩和参加练习的学生人数。

【输入样例】

```
3
li 90
bai 95
li 98
```

【输出样例】

```
98 90 2
```

第四题

试题名称：DNA 序列

时间限制： 1.0s

内存限制： 128.0MB

【问题描述】

人类基因组计划的第一阶段于 2000 年 6 月 26 日胜利结束，我国科研工作者圆满地完成了其中的 1% 的测序工作。众所周知，组成 DNA 的碱基有四种：腺嘌呤 (A)、鸟嘌呤 (G)、胞嘧啶 (C)、胸腺嘧啶 (T)。对任意两个人，其染色体上的 DNA 序列大部分相同，但总会有少数碱基对不同。这种不同是由基因的变异引起，但每个人的变异位置不尽相同。这样，对于大部分位点来说，很可能的情形是：大部分人在该位点上的碱基是一致的（没有发生变异），少数人具有不同的碱基（发生了变异）。

这就给科学家一个启示：在测序过程中，若仅仅使用一个人的样本，在很多位点上测出的结果就不具有代表性；而如果能测出多个人的序列，则有可能“整合”出一段具有人类共性的序列出来，这样更有利于研究。

例如，假设我们要检测人的某一段 DNA 序列，得到 4 人的样本：

AAAGGCCT
AGAGCTCT
AAGGATCT
AAACTTCT

整合规则：(1)取出在每个位置（即某一列）上出现次数最多的碱基作为整合后该位置上的碱基；
(2)若某个位置出现次数最多的碱基不止一种，则优先选择 A，其次是 C、G 和 T。

因此，对于上述 4 个样本，整合以后的结果为：AAAGATCT

请编写一个程序，输入一组 DNA 序列，然后对它们进行整合，并将整合以后的结果打印出来。

【输入描述】

先输入一个正整数 $N(2 \leq N \leq 10)$ ，然后再输入 N 行，每一行是一个 DNA 序列，它们具有相同的长度

【输出描述】

一个字符串，即整合以后的序列

【输入样例】

4
AAAGGCCT
AGAGCTCT
AAGGATCT
AAACTTCT

【输出样例】

AAAGATCT

第五题

试题名称：LRU 算法

时间限制： 1.0s

内存限制： 128.0MB

【问题描述】

一位考古科学家新发掘了一批恐龙蛋化石，假设有 N 个，编号分别为 $1 \sim N$ ，然后打算对它们进行研究。在科学家的实验室里，有两间房间，一间是他的办公室，一间是储物间。开始时所有的恐龙蛋化石都摆放在储物间，然后需要用到哪一个的时候，就把它拿到办公室来。在科学家的办公室有一张办公桌，桌上有 M 个盘子，如 4 个盘子，每个盘子只能摆放一个恐龙蛋化石。科学家在进行研究时，时而要研究这个，时而要研究那个，因而有一个编号序列，如 1 2 5，就是先研究 1 号化石，再研究 2 号，再研究 5 号。如果科学家需要研究某一个化石的时候，该化石正好就在办公桌的某个盘子里，那么就直接研究即可；如果该化石不在办公桌上，我们称这种情形为“缺蛋”，这时就需要站起身，去一趟隔壁的储物间，把想要的化石拿过来。但这么做有一个前提，就是办公桌上必须有空盘子，如果有空盘子，就可以把取来的化石放在某个空盘子里；而如果没有空盘子，就必须先把某一个盘子中的化石放回到储物间，从而腾出空间，然后才能把新的化石放进去。但问题是，桌子上有 M 个盘子，那么选择哪一个盘子中的化石，把它放回去（我们称为淘汰），这是需要仔细考量的。如果放回去的化石，过一会儿又要用到，那么就又得跑一趟储物间，这就浪费时间了。因此，一个朴素的想法就是

哪一个化石将来不会再用上，就把该化石拿走。但问题是，科学家自己也不知道将来会用到哪一个化石，换言之，他在做研究的时候是天马行空，想到哪儿是哪儿，因此他只知道自己当前需要用到哪一个化石，而将来的情形则不知道。在这种情形下，如何来做出比较好的淘汰决策呢？

为了减少“缺蛋”的次数，他向一位计算机科学家寻求帮助，计算机科学家向他推荐了 LRU (Least Recently Used) 算法。该算法的基本思路是，虽然将来的情形我们是不知道的，但是过去的情形是知道的，因此我们可以根据过去的情形来预期将来的情形。具体来说，如果在过去一段时间内，某个化石刚刚被用到，那么在将来一段时间内，它也很可能会再次被用到；反之，如果在过去一段时间内某个化石很久没有被用到，那么将来它也可能很久不会被用到。因此可以利用这个信息来进行淘汰。

举个例子，假设科学家研究的编号序列为 1、2、3、4、1、2、5、1、2、3、4、5（这个编号序列是事后统计出的），盘子的个数为 4，初始为空，那么采用 LRU 算法，总共会“缺蛋”8 次。即在访问最前面的 1、2、3 和 4 时，都会“缺蛋”，需要去一趟储物间。但此时由于有空闲的盘子，所以取来化石后都能直接摆进去，不需要淘汰某个化石。然后在访问接下来的 1 和 2 时，由于这两个化石已经在盘中，所以不会“缺蛋”。接下来在访问 5 号化石时，此时它不在办公桌上，然后桌上也没有空盘子了（分别装了 1、2、3 和 4 号化石），所以先要选择某个化石淘汰，根据 LRU 算法，在当前时刻，2 号是最近刚访问过的，所以要保留，1 号和 4 号其次，而 3 号是最久以前访问过的，所以就淘汰 3 号化石，把它放回到隔壁的储物间，然后把 5 号化石放到桌上的空盘里。后面是类似的，接下来的 1 和 2 不会“缺蛋”，而最后的 3、4 和 5 都会“缺蛋”，所以总共“缺蛋”8 次。

【输入描述】

输入有两行，第一行是一个正整数 M，表示盘子的个数，**其值不超过 100**。第二行是一个编号序列。

【输出描述】

输出一个正整数，表示“缺蛋”的次数。

【输入样例】

```
4
1 2 3 4 1 2 5 1 2 3 4 5
```

【输出样例】

```
8
```